UTS UNIVERSITY OF TECHNOLOGY SYDNEY

Faculty of Engineering and IT

Assignment Cover Sheet

Subject: 41277 Control Design

Assignment Number: Demonstration Practical 2 **Date Submitted:** [TODO: dd/mm/2026]

Assignment Title: State-Feedback Control

Student Name(s) and Number(s) **Tutorial Group:** 02

Jaylen Avtarovski	24767939
Rhyse Williams	24817532
Elina Kim	24949885

Declaration of Originality:

The work contained in this assignment, other than that specifically attributed to another source, is that of the author(s). It is recognised that, should this declaration be found to be false, disciplinary action could be taken and the assignments of all students involved will be given zero marks. In the statement below, I have indicated the extent to which I have collaborated with other students, whom I have named.

Acknowledge use of any copied or reworked materials (eg. text, equations, images, music, video) from generative artificial intelligence tools (GenAI) (such as ChatGPT, DALL-E) by declaring its use (you'll also need to follow the advice from your subject coordinator about whether you can use GenAI tools in assessments for that subject). Find out more about how to reference GenAI and ChatGPT in assessments.

GenAI Declaration:

[TODO: Declare any GenAI use for this report.]

Signature J.A. | E.K. | R.W. **Date** [TODO: date]

Statement of Collaboration:

[TODO: Describe how the team collaborated on Q1–Q3.]

Demonstration Practice 2

Student Number	Student Name	Overall Contribution Weighting From 0 to 100%
24767939	Jaylen Avtarovski	100%
24817532	Rhyse Williams	100%
24949885	Elina Kim	100%

Each member's individual mark is the group mark scaled by their contribution level. If not indicated, all students get the same mark.

Answers to the Assignment Questions:

Q1: State-Space Model (40 marks)

Q1.1) System Modelling

Based on [1], the QUBE-Servo 2 is modelled as

$$\dot{x}(t) = A_c\, x(t) + B_c\, u(t), \tag{1a}$$
$$y(t) = C_c\, x(t) + D_c\, u(t), \tag{1b}$$

with $n =$ **[TODO:n]** states, $m =$ **[TODO:m]** inputs and $p =$ **[TODO:p]** outputs.

[TODO: Derivation of the motor/load dynamics and the state vector $x(t)$.**]**

Q1.2) System Parameters

The physical parameters of the QUBE-Servo 2 are given in Table 1, taken from the manufacturer's user manual [3].

Table 1: QUBE-Servo 2 system parameters [3].

Symbol	Description	Value
DC motor		
R_m	Terminal resistance	$8.4\,\Omega$
k_t	Torque constant	$0.042\,\mathrm{N\,m/A}$
k_m	Motor back-emf constant	$0.042\,\mathrm{V/(rad/s)}$
J_m	Rotor inertia	$4.0 \times 10^{-6}\,\mathrm{kg\,m^2}$
L_m	Rotor inductance	$1.16\,\mathrm{mH}$
m_h	Module attachment hub mass	$0.0106\,\mathrm{kg}$
r_h	Module attachment hub radius	$0.0111\,\mathrm{m}$
J_h	Module attachment hub inertia	$0.6 \times 10^{-6}\,\mathrm{kg\,m^2}$
Inertia disc module		
m_d	Disc mass	$0.053\,\mathrm{kg}$
r_d	Disc radius	$0.0248\,\mathrm{m}$
Rotary pendulum module		
m_r	Rotary arm mass	$0.095\,\mathrm{kg}$
L_r	Rotary arm length	$0.085\,\mathrm{m}$
m_p	Pendulum link mass	$0.024\,\mathrm{kg}$
L_p	Pendulum link length	$0.129\,\mathrm{m}$
Encoders and amplifier		
	Encoder resolution (quadrature)	$0.00307\,\mathrm{rad/count}$
	Recommended output voltage	$\pm 10\,\mathrm{V}$

[TODO: Keep only the rows for the module used; add any identified parameters (e.g. viscous friction) and the identification method.**]**

Q1.3) System Model Discretisation

A sampling time of T_s = **[TODO:**T_s**]** is selected because **[TODO:** justification, e.g. relative to the closed-loop bandwidth**]**. Using a zero-order hold (`c2d`), the discrete-time model is

$$x(k+1) = A\,x(k) + B\,u(k), \tag{2a}$$
$$y(k) = C\,x(k) + D\,u(k). \tag{2b}$$

[TODO: Numerical A, B, C, D.**]**

Q2: State-Feedback Control (SFC) **(40 marks)**

Q2.1) Controllability

For this question, the controllability matrix of the continuous-time model is

$$\mathcal{C} = \begin{bmatrix} B_c & A_cB_c & \cdots & A_c^{n-1}B_c \end{bmatrix}. \tag{3}$$

[TODO: Numerical $\mathcal{C}$ (`ctrb`), its rank and justification.**]**

Q2.2) State and Input References (Equilibrium Point)

Given the desired output reference $y^\star$, the steady-state references $x^\star$ and $u^\star$ satisfy

$$\begin{bmatrix} A - I & B \\ C & D \end{bmatrix} \begin{bmatrix} x^\star \\ u^\star \end{bmatrix} = \begin{bmatrix} 0 \\ y^\star \end{bmatrix}. \tag{4}$$

[TODO: Solve for $x^\star$, $u^\star$ for your setup.**]**

Q2.3) Closed-Loop Eigenvalues

The control requirements are an overshoot below 15 % and a settling time below 5 s. The damping ratio and natural frequency follow from

$$\zeta = \frac{-\ln(OS/100)}{\sqrt{\pi^2 + \ln^2(OS/100)}}, \qquad \omega_n = \frac{4}{\zeta\, t_s}, \tag{5}$$

and the discrete-time eigenvalues from $z_i = e^{s_i T_s}$.

[TODO: Chosen s_i and z_i for all n eigenvalues, with margin.**]**

Q2.4) State-Feedback Gain

For the control law

$$u(k) = -F\big(x(k) - x^\star\big) + u^\star, \tag{6}$$

[TODO: F from `place(A,B,pz)` and verification of eig$(A - BF)$.**]**

Q2.5) Closed-Loop Implementation

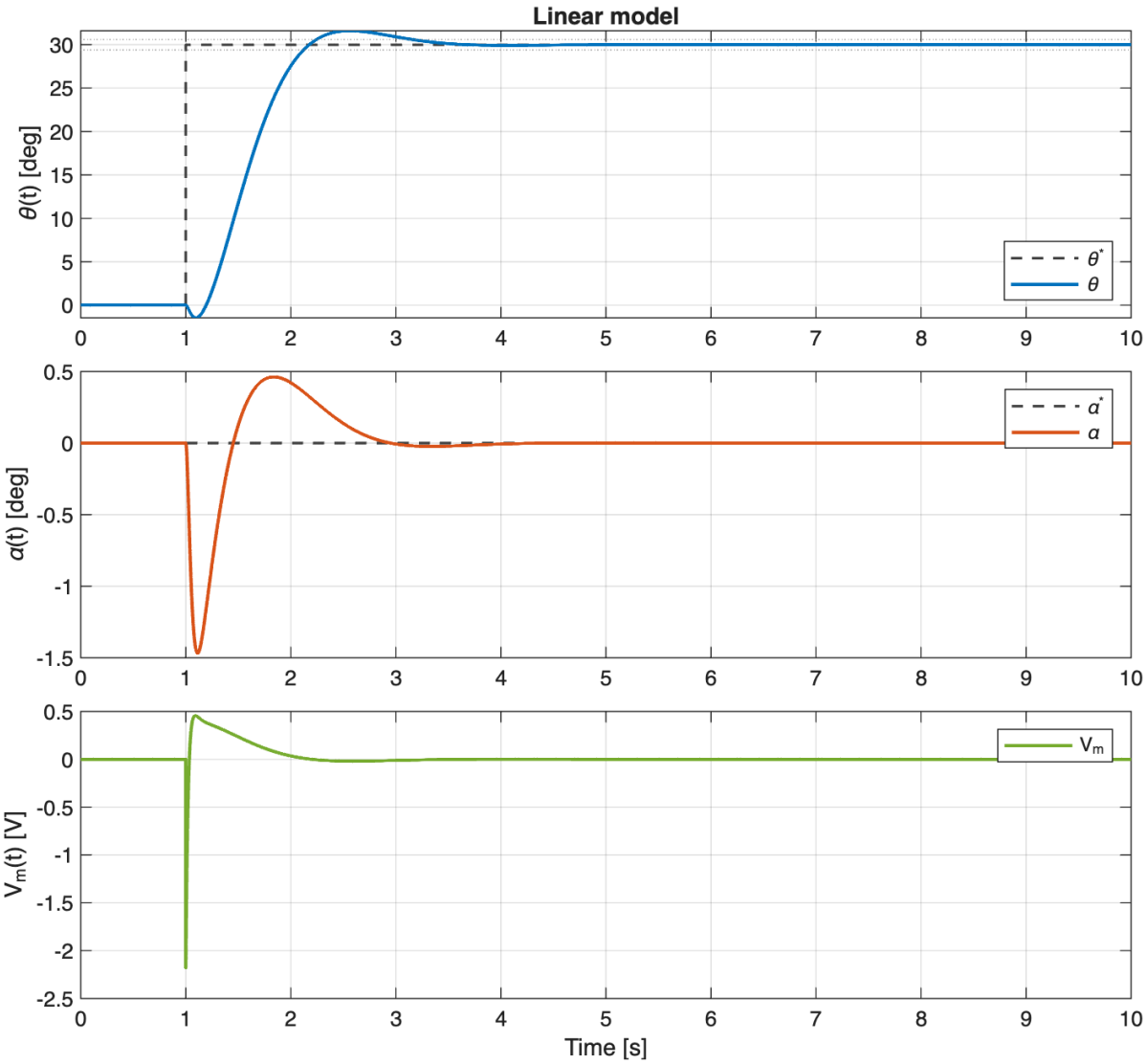


Figure 1: Closed-loop response of the linear model under state feedback.

[TODO: Comment on Figure 1: overshoot, settling time, steady-state error and control effort versus requirements. Add zooms.**]**

Q3: Luenberger Observer Design **(20 marks)**

The QUBE-Servo 2 measures only the two angles: the motor encoder gives θ and the pendulum encoder gives α, so $y = C_c x = [\theta \quad \alpha]^\mathsf{T}$. The velocities $\dot{\theta}$ and $\dot{\alpha}$ used by the state-feedback law of Q2 are not measured. They could be obtained by differencing the encoder signals, but at $T_s = 1$ ms a single encoder count (0.00307 rad $=$ 0.176° [3]) already appears as a 176°/s velocity step. Instead, a Luenberger observer reconstructs the full state from the measured outputs and the applied input, using the model of Q1.

Q3.1) Observability

With $n = 4$ states and $p = 2$ outputs, the observability matrix of the continuous-time model (1) is the $np \times n = 8 \times 4$ matrix (MATLAB: `OM = obsv(Ac,Cc)`)

$$\mathcal{O} = \begin{bmatrix} C_c \\ C_cA_c \\ C_cA_c^2 \\ C_cA_c^3 \end{bmatrix} = \left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \hline 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ \hline 0 & 54.236 & -6.179 & -1.786 \\ 0 & 167.67 & -6.107 & -5.521 \\ \hline 0 & -634.55 & 49.086 & 75.128 \\ 0 & -1256.9 & 71.453 & 209.06 \end{array}\right]. \tag{7}$$

The first four rows of (7) form the 4×4 identity matrix, because C_c selects the two angles and C_cA_c selects the two angular velocities. Hence

$$\text{rank}(\mathcal{O}) = 4 = n, \tag{8}$$

confirmed with `rank(OM)`, and the system is **observable**.

From a control viewpoint, every state shows up in the measured outputs or in their time derivatives. The velocities are exactly the rates of change of the measured angles, so the initial state can be reconstructed from a finite window of input-output data. Rank 4 is reached with only the first time derivative, which means the observer only has to infer the velocities. The ZOH discretisation does not change this: `rank(obsv(A,C))` $= 4$ as well, because T_s is far below the time scale of every open-loop mode (the open-loop eigenvalues are 0, 9.95, -4.02 and -17.63 rad/s), so pathological sampling cannot occur.

Q3.2) Observer Eigenvalues

By the separation principle [1], the eigenvalues of the observer-based closed loop are the union of the controller eigenvalues, eig$(A - BF)$, and the observer eigenvalues, eig$(A - LC)$. The observer can therefore be designed independently. It should be clearly faster than the controller, so that the estimation error has vanished before it can noticeably affect the transient set by F.

The state-feedback eigenvalues from Q2.3 are $s_{1,2} = -2 \pm j2.097$ (dominant pair: $\zeta = 0.69$, $\omega_n = 2.90$ rad/s) and $s_3 = -30$, $s_4 = -35$ rad/s. The observer eigenvalues are placed about three times faster than the fastest controller eigenvalue:

$$s_o = \{-100,\ -110,\ -120,\ -130\} \text{ rad/s}. \tag{9}$$

The reasons for this choice are:

- The slowest observer mode has a time constant $\tau = 1/100 = 10$ ms, so the estimation error settles ($\approx 4\tau$) in about 40 ms. That is 50 times faster than the dominant controller pair and about 54 times shorter than the 2.15 s closed-loop settling time.
- Real eigenvalues give a non-oscillatory error decay. They are distinct because `place` only allows each eigenvalue a multiplicity of up to rank$(C) = 2$.
- The observer is not made faster still. Faster eigenvalues need a larger gain L, which passes more encoder quantisation noise into $\hat{x}$ and increases the initial "peaking" of the velocity estimates (Figure 2). At $|s_o|T_s \leq 0.13$, each observer mode still spans more than seven samples, so the discrete design is well resolved.

Mapping these to the z-plane with $z = e^{sT_s}$, $T_s = 1$ ms (MATLAB: `po_z = exp(po_s*ts)`), gives the discrete-time observer eigenvalues

$$z_o = \{0.9048,\ 0.8958,\ 0.8869,\ 0.8781\}. \tag{10}$$

All lie on the positive real axis, inside the unit circle and closer to the origin than the controller eigenvalues $\{0.9980 \pm$

$j0.0021,\ 0.9704,\ 0.9656\}$. This confirms in the z-domain that the estimation error decays faster than the regulated response.

Q3.3) Observer Gain

The discrete-time Luenberger observer is

$$\hat{x}(k+1) = A\hat{x}(k) + Bu(k) + L\big(y(k) - \hat{y}(k)\big), \qquad \hat{y}(k) = C\hat{x}(k), \tag{11}$$

i.e. a copy of the plant model driven by the same input and corrected by the output innovation $y(k) - \hat{y}(k)$. Subtracting (11) from the plant model (2), the estimation error $e(k) = x(k) - \hat{x}(k)$ evolves as

$$e(k+1) = (A - LC)\, e(k), \tag{12}$$

independent of $u(k)$. Because $\text{eig}(A - LC) = \text{eig}(A^\mathsf{T} - C^\mathsf{T} L^\mathsf{T})$, choosing L is the dual of a state-feedback design for the pair $(A^\mathsf{T}, C^\mathsf{T})$. It is solvable because (A, C) is observable (Q3.1). In MATLAB:

```
po_z = exp([-100 -110 -120 -130]*ts);
Lo = place(A', C', po_z)';
eig(A - Lo*C)
```

which gives the $n \times p = 4 \times 2$ observer gain

$$L = \begin{bmatrix} 0.2108 & -0.0104 \\ -0.0152 & 0.2120 \\ 10.466 & -1.188 \\ -2.168 & 10.861 \end{bmatrix}. \tag{13}$$

Evaluating the eigenvalues of the error-dynamics matrix gives

$$\text{eig}(A - LC) = \{0.9048,\ 0.8958,\ 0.8869,\ 0.8781\}, \tag{14}$$

identical to (10), so the observer has the desired eigenvalues.

The structure of L has a clear physical reading. Column 1 weights the θ innovation $\theta - \hat{\theta}$, and column 2 weights the α innovation $\alpha - \hat{\alpha}$. Each angle estimate is corrected mainly by its own measurement (0.21 per sample, on the diagonal of the first two rows). Each velocity estimate is corrected by about $10.5\ \text{s}^{-1}$ per radian of its own angle innovation, which works like a model-based, filtered differentiator. The small off-diagonal terms account for the coupling between the arm and the pendulum in A.

Q3.4) Closed-Loop Implementation

The observer was implemented in the provided Simulink models (`sim_DP2_2026_SFC_Linear.slx` and `sim_DP2_2026_SFC_Nonlinear.slx`) inside the MATLAB Function block *Digital Controller*, which is sampled at $T_s = 1$ ms. The controller now feeds back the estimated state:

$$u(k) = -F\big(\hat{x}(k) - x^\star\big) + u^\star. \tag{15}$$

The core of the block (with `run_observer = 1`) is:

```
x_hat = x_hat_k; % estimate used at sample k
xu_ref = [A-eye(n) B; C zeros(p,m)] \ [zeros(n,1); y_ref];
x_ref = xu_ref(1:n); u_ref = xu_ref(n+1:n+m); % Q2.2
u_k = -F*(x_hat_k - x_ref) + u_ref;
u_k = min(max(u_k, -10), 10); % +/-10 V amplifier limit
y_hat = C*x_hat_k; % Luenberger observer (Q3.3)
x_hat_k = A*x_hat_k + B*u_k + Lo*(y_k - y_hat);
```

The observer is updated with the *saturated* voltage that is actually applied to the motor. Feeding it the unsaturated command would make its model input differ from the plant input whenever the amplifier limits, and the estimate would drift.

The controller was applied to both the linearised model (Q1.1) and the nonlinear rotary-pendulum model. The nonlinear model is the same Euler–Lagrange model before linearisation, with $\sin\alpha$, $\cos\alpha$, $\dot\theta^2$, $\dot\alpha^2$ and $\dot\theta\dot\alpha$ terms retained, implemented in `rip_nonlinear_dynamics.m`. Both runs use the arm reference $\theta^\star = 30^\circ$ applied as a step at $t = 1$ s and $\alpha^\star = 0$ (upright), for which Q2.2 gives $x^\star = [\theta^\star\ 0\ 0\ 0]^\mathsf{T}$ and $u^\star = 0$. As Remark 5 asks, the plant starts at rest upright, $x(0) = 0$, while the observer is deliberately initialised with

$$\hat{x}(0) = \begin{bmatrix} 10^\circ & -5^\circ & 0 & 0 \end{bmatrix}^\mathsf{T} \neq x(0). \tag{16}$$

All signals were exported to the workspace and plotted with `plot_closed_loop.m`. The script `run_all_cases.m` regenerates every figure.

Observer convergence. Figure 2 compares the true and estimated states of the nonlinear model, with a zoom on the first 0.1 s. Figure 3 shows the estimation error on a logarithmic scale. The angle estimates converge onto the measured angles within a few milliseconds, and both angle errors stay below 0.1° after 54 ms in both the linear and nonlinear cases. This agrees with the ≈ 40 ms predicted from the slowest observer eigenvalue in Q3.2. After the first few samples, the four error curves in Figure 3 are straight lines on the logarithmic axis, which is the exponential decay of (12). The final error is about 10^{-12}, at the level of numerical round-off.

The zoom in Figure 2 also shows the price of a fast observer. The velocities are not measured, so the initial 10° and -5° angle innovations are converted through the large velocity gains of L into velocity-estimate errors that peak at $389^\circ/\text{s}$ ($\dot\theta$) and $254^\circ/\text{s}$ ($\dot\alpha$) about 8–9 ms after start-up. This is the typical peaking of a high-gain observer, and it is why the observer eigenvalues were not pushed further left.

Closed-loop performance. Figures 4 and 5 show the response of the observer-based controller on the linear and nonlinear models. Table 2 compares it with the full-state feedback of Q2.5. Because $u(k)$ is computed from $\hat{x}(k)$, the wrong initial estimate acts like an initial-condition disturbance. At $k = 0$ the controller "sees" the pendulum 5° off upright and commands $u(0) = 8.98$ V. The voltage then saturates at -10 V for 21 ms while the estimate converges. This briefly kicks the pendulum to $|\alpha| = 5.8^\circ$, compared with 1.5° under full-state feedback, and moves the arm by about 12° before the controller recovers. Under full-state feedback, nothing happens before the step.

Once the estimation error has converged (well before the step at $t = 1$ s), $\hat{x}(k) \approx x(k)$ and, by the separation principle, the response is governed only by $\text{eig}(A - BF)$. The step response is accordingly almost identical to Q2.5: $4.6\,\%$ overshoot (requirement $< 15\,\%$), 2.16 s settling time to the $2\,\%$ band (requirement < 5 s) and negligible steady-state error. The slightly lower overshoot than in Q2.5 comes from the arm still returning from the start-up transient when the step is applied. All requirements are met in both models. Saturation briefly breaks the linearity that the separation principle assumes. Because the observer uses the applied (saturated) input, the estimation-error dynamics (12) are unaffected and the estimate stays correct throughout.

Table 2: Simulated step-response metrics ($\theta^\star = 30^\circ$ at $t = 1$ s; overshoot and $2\,\%$ settling time measured from the step).

Model	**Feedback**	**OS [%]**	t_s **[s]**	**max** $\lvert\alpha\rvert$ **[deg]**	**max** $\lvert V_m\rvert$ **[V]**
Linear	$x(k)$ (Q2.5)	5.32	2.15	1.47	2.18
Nonlinear	$x(k)$ (Q2.5)	5.30	2.15	1.47	2.18
Linear	$\hat{x}(k)$ (Q3.4)	4.64	2.16	5.82	10.0 (sat.)
Nonlinear	$\hat{x}(k)$ (Q3.4)	4.59	2.16	5.84	10.0 (sat.)
Requirement		< 15	< 5	–	≤ 10

The linear and nonlinear responses differ by less than 0.1 percentage points of overshoot. The pendulum never

leaves $|\alpha| < 6°$, where $\sin\alpha \approx \alpha$ is accurate to better than $0.2\,\%$, so the linearised model on which both F and L are based remains a faithful description of the plant.

On the real QUBE-Servo 2, two practical points follow from these results. First, the start-up kick can be avoided by initialising $\hat{\theta}(0)$ and $\hat{\alpha}(0)$ with the first encoder readings, since only the unmeasured velocities then need to be estimated. Second, the simulated steady-state error is zero only because the model is exact. Unmodelled Coulomb friction and parameter mismatch on the hardware would leave an offset in θ that this proportional state feedback cannot remove without integral action.

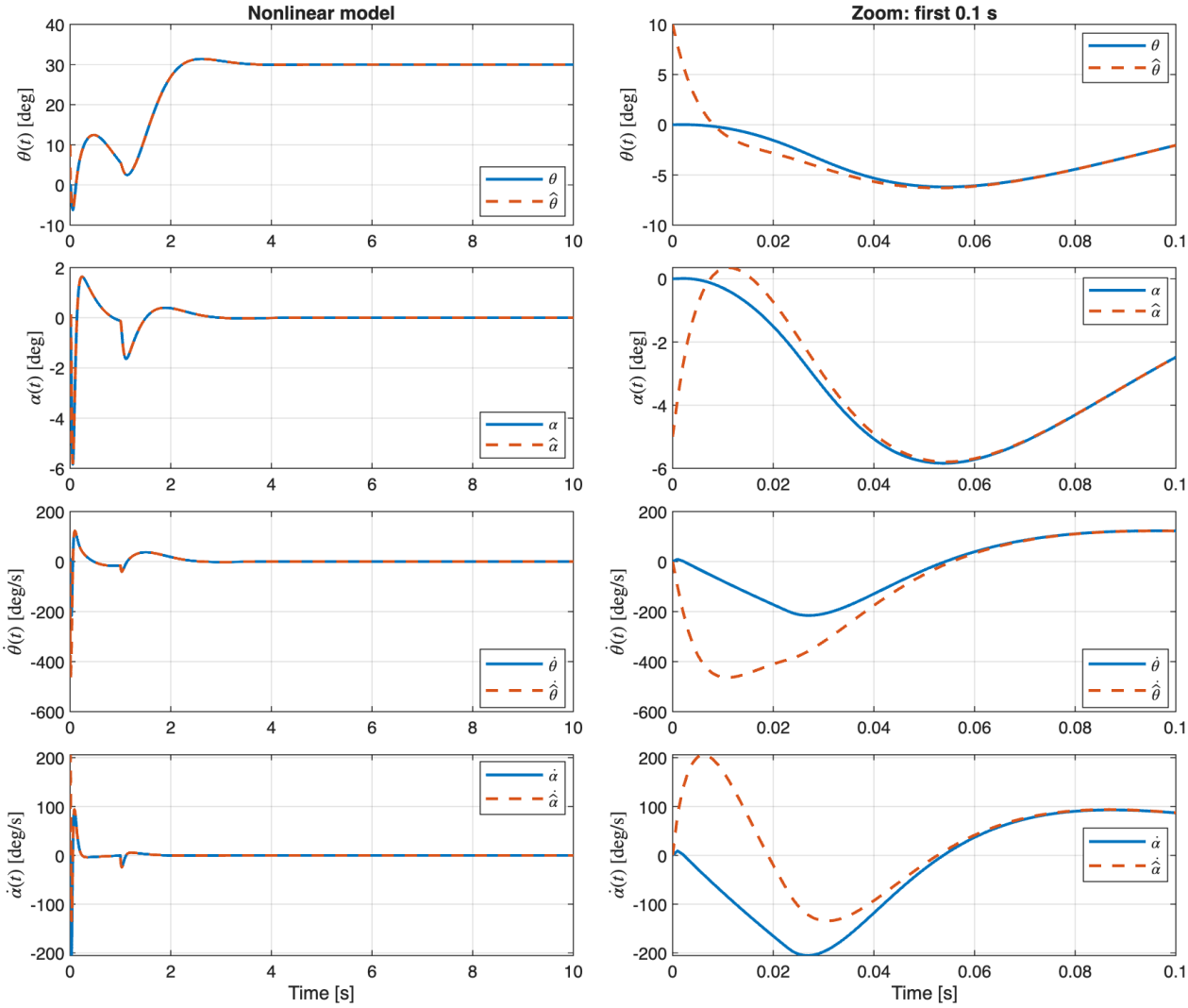


Figure 2: True (x, solid) and estimated ($\hat{x}$, dashed) states of the nonlinear model with $\hat{x}(0) = [10°\ \ -5°\ 0\ 0]^\mathsf{T} \neq x(0) = 0$. Left: full simulation; right: zoom on the first 0.1 s showing observer convergence.

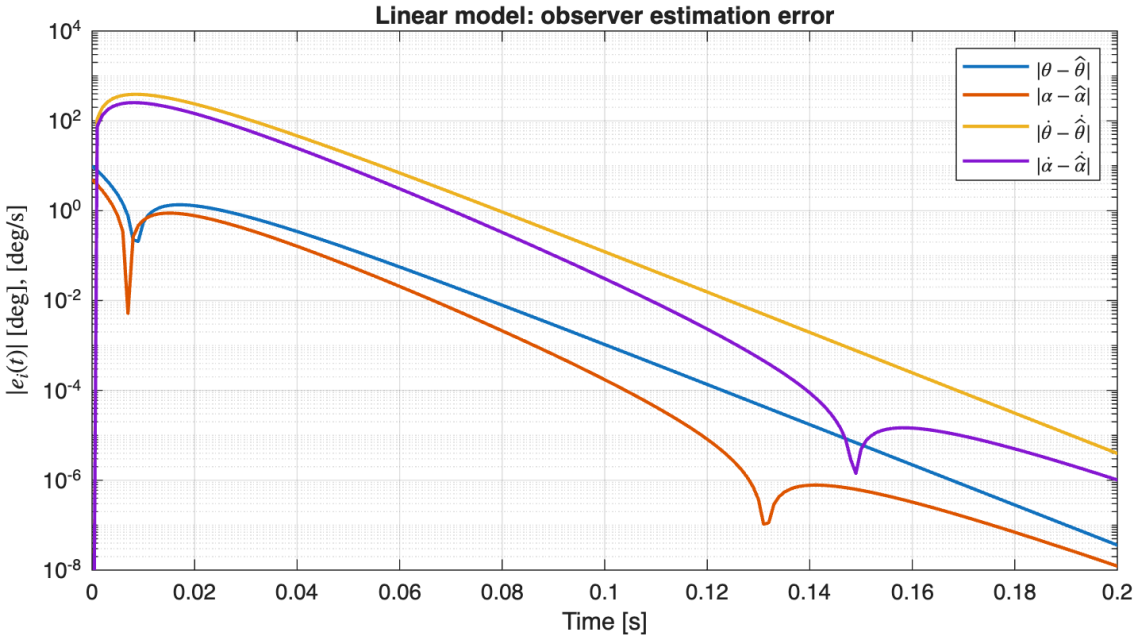


Figure 3: Magnitude of the estimation error $e = x - \hat{x}$ (linear model, logarithmic scale) during the first 0.2 s; the straight-line decay reflects the exponential error dynamics $e(k+1) = (A - LC)e(k)$.

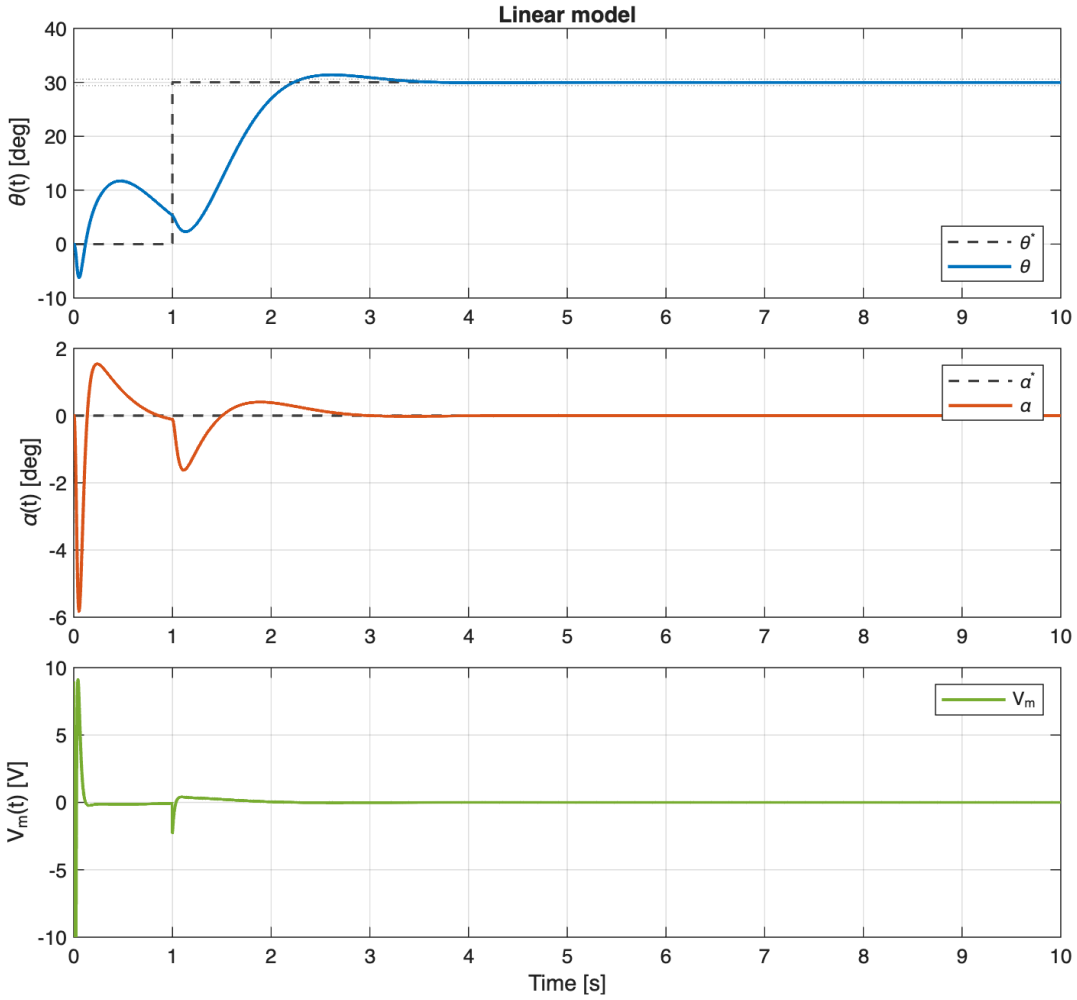


Figure 4: Observer-based state feedback on the linear model: arm angle θ with reference $\theta^\star$ (dotted lines: $\pm 2\,\%$ band), pendulum angle α, and motor voltage V_m.

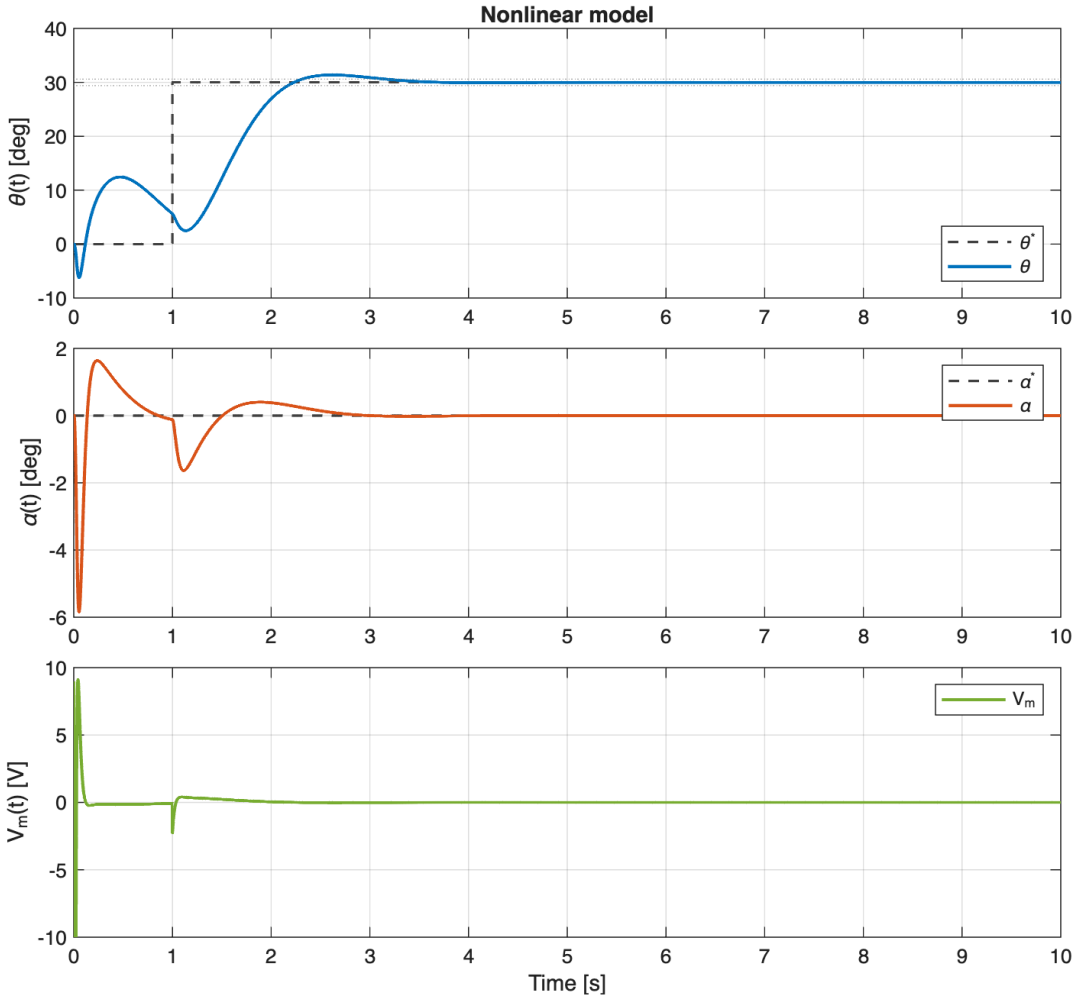


Figure 5: Observer-based state feedback on the nonlinear model: arm angle θ with reference $\theta^\star$ (dotted lines: $\pm 2\,\%$ band), pendulum angle α, and motor voltage V_m.

References

[1] R. P. Aguilera, *Lecture Notes on State-Feedback Control*, University of Technology Sydney, Australia, 2026.

[2] B. Messner and D. Tilbury, *Control Tutorials for MATLAB and Simulink*, University of Michigan, USA. [Online]. Available: http://ctms.engin.umich.edu/CTMS

[3] Quanser Inc., *QUBE-Servo 2 User Manual*, v1.0, Markham, ON, Canada.